

INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA
DE MINAS GERAIS - *CAMPUS* SÃO JOÃO EVANGELISTA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Valdir de Souza Carvalho Neto

ENP 06 - Comunicação entre Processos e Concorrência

Sistemas Operacionais

São João Evangelista

2026

SUMÁRIO

1. Estudo de caso Sistema de Impressão.....	3
1.1. Por que a impressora é considerada um recurso compartilhado?.....	3
1.2. O que pode acontecer se não houver sincronização?.....	3
1.3. Como um mutex poderia resolver o problema?.....	3
1.4. Existe necessidade de região crítica? Justifique.....	3
2. Estudo de Caso Loja Virtual.....	3
2.1. Que problema de concorrência pode ocorrer?.....	3
2.2. Como a condição de corrida afeta o estoque?.....	4
2.3. Qual mecanismo de sincronização você utilizaria?.....	4
2.4. Qual seria a consequência para a empresa caso não exista controle adequado?.....	4
3. Explique o conceito de Comunicação entre Processos (IPC) e apresente três mecanismos utilizados pelos sistemas operacionais.....	4
4. Defina condição de corrida e apresente um exemplo prático.....	4
5. Explique a diferença entre processo e thread.....	4
6. O que é uma região crítica? Por que ela precisa ser protegida?.....	5
7. Explique o conceito de exclusão mútua.....	5
8. Quais são os requisitos de uma solução correta para exclusão mútua?.....	5
9. Descreva o funcionamento do Algoritmo de Peterson.....	6
10. Explique o funcionamento dos semáforos e suas operações fundamentais.....	6
11. Explique a técnica de Dormir e Acordar e sua vantagem em relação à espera ocupada.....	6
12. Defina deadlock.....	7
13. Explique as quatro condições de Coffman.....	7
14. Descreva o Algoritmo do Banqueiro.....	7
15. Apresente duas formas de evitar deadlocks.....	7
16. Explique por que a sincronização é importante em sistemas multicore.....	8

1. Estudo de caso Sistema de Impressão

Uma empresa possui apenas uma impressora compartilhada por diversos usuários. Vários processos enviam documentos para impressão simultaneamente.

1.1. Por que a impressora é considerada um recurso compartilhado?

Resposta: A impressora é um recurso compartilhado porque vários processos enviam documentos para ela simultaneamente, exigindo controle para evitar problemas de sincronização.

1.2. O que pode acontecer se não houver sincronização?

Resposta: Sem a devida sincronização, pode ocorrer a chamada Condição de Corrida (*Race Condition*). Nessa situação, os processos entram em conflito e o resultado da operação passa a depender da ordem de execução, gerando inconsistências.

1.3. Como um mutex poderia resolver o problema?

Resposta: A exclusão mútua (mutex), - versão simplificada do semáforo que é utilizada quando a capacidade de fazer contagens não é necessária - resolveria o problema ao garantir que apenas um único processo tenha acesso à região crítica por vez, mantendo a consistência dos dados e evitando os conflitos.

1.4. Existe necessidade de região crítica? Justifique.

Resposta: Sim, existe a necessidade de uma região crítica. A região crítica é o trecho específico do programa que realiza o acesso aos dados ou recursos compartilhados, e sua execução sob controle é indispensável para evitar inconsistências.

2. Estudo de Caso Loja Virtual

Uma loja virtual vende um produto que possui apenas uma unidade em estoque. Dois clientes realizam a compra exatamente no mesmo instante. processos enviam documentos para impressão simultaneamente.

2.1. Que problema de concorrência pode ocorrer?

Resposta: O principal problema que pode ocorrer é a Condição de Corrida. Ela acontece quando dois ou mais processos acessam um recurso simultaneamente.

2.2. Como a condição de corrida afeta o estoque?

Resposta: O estoque é afetado porque o resultado da transação passa a depender da ordem de execução dos clientes, o que gera comportamentos imprevisíveis e dados inconsistentes.

2.3. Qual mecanismo de sincronização você utilizaria?

Resposta: A solução adequada seria implementar mecanismos de Exclusão Mútua, utilizando, por exemplo, Semáforos, primitivas *Sleep/Wakeup*, Espera Ocupada e Mutex.

2.4. Qual seria a consequência para a empresa caso não exista controle adequado?

Resposta: A consequência seria a ocorrência de dados inconsistentes, como a venda indevida de mais unidades do que o existente em estoque por falta de um acesso controlado. Gerando assim, transtornos para a empresa e para os clientes.

3. Explique o conceito de Comunicação entre Processos (IPC) e apresente três mecanismos utilizados pelos sistemas operacionais.

Resposta: A comunicação entre processos (IPC) é fundamental em sistemas modernos para que os processos possam trocar informações e sincronizar ações de maneira estruturada e sem utilizar interrupções, tornando o processo eficiente. Três mecanismos voltados para essa exclusão e sincronização são: Espera Ocupada (*Busy Waiting*), Primitivas *Sleep/Wakeup* e Semáforos.

4. Defina condição de corrida e apresente um exemplo prático.

Resposta: A condição de corrida ocorre quando múltiplos processos ou threads acessam dados compartilhados simultaneamente, e o resultado da operação depende estritamente da ordem de execução desses processos. Um exemplo prático é um sistema bancário onde o saldo da conta é R\$1000,00 e duas threads tentam ler o saldo e gravar um saque de R\$700,00 simultaneamente, o que resultará em um saldo incorreto.

5. Explique a diferença entre processo e thread.

Resposta: Um processo é uma unidade de execução pesada e independente gerenciada pelo sistema operacional, que recebe seu próprio espaço de memória

estruturalmente isolado, garantindo que falhas locais não corrompam outros programas, mas tornando sua criação e a troca de informações mais custosas para a CPU. Por outro lado, a thread é uma unidade de execução muito mais leve que existe *dentro* de um processo; múltiplas threads compartilham integralmente a mesma memória e os recursos do processo pai, o que garante uma alternância de tarefas extremamente rápida, mas exige a implementação de mecanismos rigorosos de sincronização (como exclusão mútua e semáforos) para evitar dados inconsistentes causados por condições de corrida. Para ilustrar, podemos pensar no Microsoft Word aberto no computador como um único processo; as tarefas simultâneas rodando lá dentro - como capturar a digitação, verificar a ortografia e realizar o salvamento automático - são as threads.

6. O que é uma região crítica? Por que ela precisa ser protegida?

Resposta: A região crítica é o trecho de um programa onde os acessos a variáveis e recursos compartilhados são realizados. Ela precisa ser protegida para manter a consistência dos dados e impedir que mais de um processo modifique os mesmos recursos ao mesmo tempo.

7. Explique o conceito de exclusão mútua.

Resposta: A exclusão mútua é o principal mecanismo de sincronização utilizado para evitar que a Condição de Corrida (*Race Condition*) aconteça. Ela garante que apenas um único processo (ou thread) tenha permissão para entrar e executar a sua "região crítica" (o trecho de código que manipula dados compartilhados) por vez. Se o Processo A está na região crítica, qualquer outro processo (como o Processo B) que tente acessar esse mesmo recurso será bloqueado e forçado a esperar (gerando uma fila de espera ou *overhead*) até que o Processo A saia da região crítica. O objetivo final é manter a consistência dos dados sob controle e evitar comportamentos imprevisíveis.

8. Quais são os requisitos de uma solução correta para exclusão mútua?

Resposta:

- Dois processos nunca podem estar dentro de suas regiões críticas simultaneamente.
- Nenhuma suposição deve ser feita em relação à velocidade de processamento ou ao número de CPUs disponíveis.
- Um processo posicionado fora da região crítica não pode causar bloqueio a um outro processo.

- Um processo nunca deve esperar de forma eterna para entrar em sua região crítica, não sofrendo inanição.

9. Descreva o funcionamento do Algoritmo de Peterson.

Resposta: É uma solução de *software* destinada a dois processos, elaborada para assegurar exclusão mútua, progresso e espera limitada. Antes de acessar a região crítica, o processo assinala seu interesse ao chamar a variável "entra-região", sendo que a entrada na região crítica ocorre apenas quando há o retorno seguro indicando que não haverá conflitos. Após a finalização de sua tarefa, o processo chama o comando "deixa-região".

10. Explique o funcionamento dos semáforos e suas operações fundamentais.

Resposta: Semáforos são variáveis especiais introduzidas para resolver problemas de sincronização mais complexos, como o "sinal de acordar perdido" visto no modelo Produtor-Consumidor. Eles garantem exclusão mútua, sincronização de eventos e evitam que processos morram de fome (*starvation*) ou entrem em impasses (*deadlocks*). Eles funcionam baseados em duas operações fundamentais:

- Wait (P) / down: Esta operação decrementa o valor do semáforo. Se o valor indicar que não há recursos disponíveis, o processo que executou o *Wait* é imediatamente bloqueado e colocado para esperar.
- Signal (V) / up: Esta operação incrementa o valor do semáforo, indicando que um recurso foi liberado. Ao fazer isso, ela tem a capacidade de "acordar" um processo que estava bloqueado aguardando por aquele recurso

11. Explique a técnica de Dormir e Acordar e sua vantagem em relação à espera ocupada.

Resposta: A técnica de Dormir (*sleep*) e Acordar (*wakeup*) é uma primitiva de comunicação simples baseada em bloqueio. Em vez de tentar acessar um recurso repetidamente, um processo avalia uma condição; se ele não puder prosseguir (por exemplo, tentar ler um buffer que está vazio), ele chama a operação *sleep* e é bloqueado. Ele permanecerá "dormindo" até que outro processo altere a condição do sistema e execute a operação *wakeup* para despertá-lo. A vantagem dessa técnica sobre a Espera Ocupada (*Busy Waiting*) é a eficiência no uso do *hardware*. Na Espera Ocupada, o processo fica travado em um laço de repetição contínuo (como um *while* testando uma variável), gastando tempo de processamento da CPU à toa. A técnica de *Sleep/Wakeup* evita esse desperdício de CPU, pois

o processo adormecido é retirado de execução pelo sistema operacional até ser chamado novamente.

12. Defina deadlock.

Resposta: Deadlock é a situação de sistema em que os processos ficam bloqueados indefinidamente. Isso se concretiza quando cada processo fica segurando um recurso do sistema e passa a aguardar por outro, formando um ciclo ou impasse.

13. Explique as quatro condições de Coffman.

Resposta:

- Condição de exclusão mútua: Determina que os recursos só podem ser utilizados por um processo de cada vez.
- Condição de posse e espera: Processos que já estão retendo recursos possuem o direito de solicitar a posse de novos recursos.
- Condição de não preempção: Afirma que recursos que foram previamente concedidos a um processo não podem ser tomados dele à força.
- Condição de espera circular: Caracteriza-se por uma cadeia circular envolvendo dois ou mais processos, onde cada um deles permanece aguardando pelo recurso retido pelo próximo componente da cadeia.

14. Descreva o Algoritmo do Banqueiro.

Resposta: É um algoritmo elaborado por Dijkstra com o principal objetivo de evitar a ocorrência de deadlocks no sistema. Sua lógica consiste em verificar, antes de conceder qualquer recurso, se aquela alocação manterá o sistema operando em um estado seguro. Consequentemente, o sistema somente aprova solicitações que o preservem nesse estado seguro.

15. Apresente duas formas de evitar deadlocks.

Resposta: A Evasão, implementada na prática através de métodos como o Algoritmo do Banqueiro. A Prevenção e Impedimento, que é focada na eliminação estratégica de pelo menos uma das condições essenciais de Coffman para que o deadlock não ocorra.

16. Explique por que a sincronização é importante em sistemas multicore.

Resposta: Em sistemas compostos por múltiplas CPUs, práticas simplistas de sincronização, como desabilitar interrupções, não impedem que os outros núcleos acessem a memória de forma simultânea. A sincronização é vital porque uma boa solução de programação concorrente não pode fazer suposições atreladas ao número de processadores (CPUs) do *hardware* e precisa garantir que dois processos nunca acessem a mesma região crítica ao mesmo tempo em nenhuma circunstância arquitetural.